



Universidade do Minho
Escola de Engenharia

Ambiente Multi-Runner

Sistemas Operativos

Alexandre Gabriel Silva Machado (A106801)
~~Daniel Ramos de Almeida Marcos Costa (A108708)~~
Simão Pedro Ribeiro Santos (A109767)

10 de maio de 2026
Braga, Portugal

Índice de conteúdos

1. Introdução	1
1.1. Contexto e objetivos do sistema Multi-Runner	1
1.2. Visão global da solução e principais decisões de desenho	1
2. Arquitetura do sistema e gestão de processos	1
2.1. Modelo cliente-servidor: <i>runner</i> e <i>controller</i>	1
2.2. Separação de responsabilidades entre módulos	2
2.3. Execução de comandos e herança de descritores	2
3. Parsing, redirecionamentos e pipelines	3
3.1. Interpretação dos comandos recebidos pelo runner	3
3.2. Redirecionamento de descritores com <code>dup2</code>	3
3.3. Criação de pipelines com <code>pipe</code> , <code>fork</code> e <code>execvp</code>	3
4. Mecanismos de comunicação entre processos	4
4.1. Comunicação via FIFOs	4
4.2. Protocolo de mensagens entre runner e controller	4
4.3. Escritas atômicas no FIFO	4
5. Estrutura de dados e políticas de escalonamento	5
5.1. Representação dos comandos em espera e em execução	5
5.2. Implementação das políticas FCFS, Random e Fair	5
5.3. Consulta de estado com a opção <code>-c</code>	6
6. Avaliação experimental	6
6.1. Testes automatizados de parsing e redirecionamento	6
6.2. Testes de concorrência e políticas de escalonamento	7
6.3. Teste de stress e análise do turnaround time	8
6.4. Metodologia de desenvolvimento dos testes com apoio de IA	8
7. Conclusão	8
7.1. Síntese dos resultados obtidos	8
7.2. Limitações e possíveis melhorias	9
Anexos	10
Anexo A – Teste de parsing e redireccionamentos	10
Anexo B – Teste da política FCFS e limite de concorrência	11
Anexo C – Teste de stress ao canal IPC	12
Anexo D – Teste da política de escalonamento Fair	13
Anexo E – Teste da política Random	14

Índice de tabelas

Tabela 1: Turnaround time de 5 comandos idênticos (sleep 4) submetidos concorrentemente.	7
--	---

1. Introdução

1.1. Contexto e objetivos do sistema Multi-Runner

O presente relatório descreve a concepção e implementação de um sistema de orquestração de comandos para sistemas baseados em UNIX/POSIX (garantindo total compatibilidade com o ambiente Linux exigido), designado por “Ambiente Multi-Runner”. O objetivo do projeto consistiu em desenvolver uma solução capaz de receber pedidos de execução de vários utilizadores em simultâneo e controlar a sua execução através de um servidor central.

Para evitar a execução descontrolada de processos, o sistema respeita um limite máximo de comandos em paralelo, definido no arranque do *controller*. Quando esse limite é atingido, os novos pedidos ficam numa fila de espera até poderem ser executados. A escolha dos comandos a iniciar é feita por uma política de escalonamento, tendo sido implementadas as políticas *First-Come, First-Served* (FCFS), Aleatória (*Random*) e a Justa (*Fair*). O sistema suporta ainda a consulta do estado atual dos comandos e o encerramento controlado do servidor.

1.2. Visão global da solução e principais decisões de desenho

A solução segue um modelo cliente-servidor, dividindo-se em dois programas principais: o `controller`, responsável pela gestão dos pedidos, e o `runner`, responsável pela interação com o utilizador e pela execução dos comandos.

Em conformidade com o enunciado, a execução dos comandos foi mantida no `runner`, ficando o `controller` responsável apenas pelo escalonamento e pela gestão de estado. Assim, o servidor apenas gere filas, estados e notificações, enquanto cada `runner` executa localmente o comando que submeteu, recorrendo a chamadas ao sistema como `fork` e `execvp`. Esta abordagem simplifica o isolamento entre o servidor e os comandos executados e permite que o *output* do comando seja apresentado diretamente no terminal do utilizador.

A comunicação entre `runner` e `controller` é feita através de FIFOs, usando mensagens de tamanho fixo. Além disso, foi implementado um módulo de *parsing* que interpreta redirecionamentos de entrada, saída e erro (`<`, `>`, `2>`) e *pipelines* (`|`). A execução destes operadores é feita através da criação de processos, *pipes* anónimos e redirecionamento de descritores com `dup2`, sem recorrer à função `system()` nem à execução indireta através da *shell*.

2. Arquitetura do sistema e gestão de processos

2.1. Modelo cliente-servidor: *runner* e *controller*

O sistema segue uma arquitetura cliente-servidor composta por dois programas principais, que comunicam através de FIFOs. O `controller` atua como servidor central, recebendo os pedidos de execução, mantendo a informação dos comandos em espera e em execução, e garantindo que o limite de paralelismo definido no arranque é respeitado.

O `runner` atua como cliente, servindo de interface entre o utilizador e o sistema. Antes de executar um comando, envia um pedido ao `controller` e aguarda pela respetiva autorização. Após receber essa autorização, o próprio `runner` executa o comando. Desta forma, o `controller` fica responsável apenas pela gestão de estado e pelo escalonamento, enquanto a execução fica distribuída pelos vários processos `runner`.

2.2. Separação de responsabilidades entre módulos

A implementação ficou organizada em módulos principais nos diretórios `src/runner/`, `src/controller/` e `src/common/`, com os respetivos ficheiros de cabeçalho (*headers*) em `include/`. Esta divisão separa o código específico do cliente, o código específico do servidor e os módulos partilhados pelos dois programas.

No diretório `common/`, a comunicação entre processos foi concentrada no módulo `ipc.c`. Este módulo reúne as operações sobre FIFOs, encapsulando chamadas ao sistema como `mkfifo`, `open`, `read`, `write` e `close`. Assim, o restante código não precisa de lidar diretamente com todos os detalhes de criação, abertura, leitura e escrita nos *pipes* com nome.

No diretório `controller/`, a lógica do servidor foi dividida entre o ciclo principal, o tratamento de pedidos em `controller_handlers.c` e as estruturas de dados em `scheduler.c`. A gestão das filas é feita através do tipo opaco `job_queue_t`, que isola as operações de inserção, remoção e seleção do próximo comando. Desta forma, o ciclo principal do `controller` mantém-se focado na receção de mensagens, no escalonamento e na atualização do estado dos comandos.

Foi também tratado o encerramento controlado do sistema. Quando o `controller` recebe um pedido de terminação, deixa de aceitar novos comandos para execução e responde aos novos *runners* com `SHUTDOWN_ACK`, evitando que estes fiquem bloqueados à espera de autorização.

No diretório `runner/`, a interpretação dos argumentos da linha de comandos ficou isolada em `runner_cli.c`. A execução efetiva dos comandos foi delegada ao módulo `executor.c`, que trata da criação de processos, da aplicação de redirecionamentos e da construção de pipelines. Com esta separação, o programa principal do `runner` fica responsável sobretudo pela comunicação com o `controller` e pela interação com o utilizador.

2.3. Execução de comandos e herança de descritores

Uma decisão importante foi executar os comandos no `runner`, e não no `controller`. Esta opção tem duas vantagens principais.

Em primeiro lugar, melhora o isolamento entre o servidor e os comandos submetidos. Se um comando terminar com erro, apenas o processo associado a essa execução é afetado, enquanto o `controller` continua ativo e disponível para processar outros pedidos.

Em segundo lugar, aproveita a herança de descritores após o `fork()`. Quando o `runner` cria um processo filho, este herda os descritores de ficheiro do processo pai, incluindo `stdin`, `stdout` e `stderr`. Assim, comandos como `ls` ou `echo` escrevem diretamente no terminal do utilizador, sem ser necessário o `controller` reenviar o output.

A execução final é feita com `execvp()`, que substitui a imagem do processo filho pelo programa pedido. Quando existem redirecionamentos ou *pipes*, os descritores são ajustados com

`dup2()` antes da chamada a `execvp()`, garantindo que cada processo filho executa com a entrada e a saída pretendidas.

3. Parsing, redirecionamentos e pipelines

3.1. Interpretação dos comandos recebidos pelo runner

Para evitar o uso de `system()` ou de uma *shell* externa, o projeto inclui um módulo de *parsing* próprio. Este módulo recebe a *string* do comando passada ao `runner` e divide-a em comandos, argumentos e operadores.

Um ponto importante foi suportar operadores de redirecionamento (`<`, `>`, `2>`) e de pipeline (`|`) mesmo quando aparecem sem espaços à volta, como em `cat<input.txt|wc -l>out.txt`. O *parser* transforma esta *string* numa estrutura interna com os comandos, os respetivos argumentos e os ficheiros associados aos redirecionamentos. Essa estrutura é depois usada pelo módulo de execução.

3.2. Redirecionamento de descritores com dup2

Os redirecionamentos são aplicados no processo filho, depois do `fork()` e antes da chamada a `execvp()`. O executor começa por abrir os ficheiros necessários com `open()`, usando as *flags* adequadas: por exemplo, `O_RDONLY` para entrada e `O_CREAT | O_TRUNC | O_WRONLY` para saída.

Depois, os descritores são redirecionados com `dup2()`. Num redirecionamento de saída, por exemplo, `dup2(fd_out, STDOUT_FILENO)` faz com que a saída padrão passe a apontar para o ficheiro aberto. O mesmo princípio é aplicado à entrada padrão, com `STDIN_FILENO`, e ao erro padrão, com `STDERR_FILENO`. Assim, quando `execvp()` é chamado, o novo programa já herda os descritores configurados e lê ou escreve no destino correto.

3.3. Criação de pipelines com pipe, fork e execvp

Para executar comandos ligados por pipelines (`|`), o sistema cria *pipes* anónimos com `pipe()`. Numa cadeia com *N* comandos, são necessários *N-1* *pipes*.

A execução é feita de forma iterativa. Para cada comando, o `runner` cria um processo filho com `fork()`. No filho, se existir um comando anterior, a entrada padrão é ligada à extremidade de leitura do *pipe* anterior. Se existir um comando seguinte, a saída padrão é ligada à extremidade de escrita do *pipe* atual. Estas ligações são feitas com `dup2()`.

Depois de configurar os descritores, o processo filho fecha os descritores de que já não precisa e chama `execvp()`, passando a executar o comando pretendido. O processo pai fecha também os descritores que já não usa, para evitar manter extremidades de *pipes* abertas desnecessariamente.

Após criar todos os processos da *pipeline*, o `runner` usa `wait()` para aguardar pela terminação dos filhos. Só depois comunica ao `controller` que o comando terminou, garantindo que a tarefa apenas é dada como concluída quando toda a *pipeline* acaba.

4. Mecanismos de comunicação entre processos

4.1. Comunicação via FIFOs

O `controller` e os vários processos `runner` são iniciados de forma independente, podendo estar em terminais distintos. Como não existe necessariamente uma relação pai-filho entre eles, os *pipes* anônimos não são adequados para a submissão inicial de pedidos. Por isso, a comunicação entre `runner` e `controller` foi implementada com FIFOs.

O `controller` cria um FIFO público com `mkfifo()`, conhecido pelos runners, e usa-o para receber pedidos. Cada `runner` abre esse FIFO em modo de escrita para enviar mensagens ao servidor. Esta organização corresponde a um modelo com vários produtores e um consumidor: vários *runners* submetem pedidos, enquanto o `controller` centraliza a receção e tratamento dessas mensagens.

4.2. Protocolo de mensagens entre runner e controller

Para uniformizar a comunicação, foi definida a estrutura `RpcMessage`, enviada através do FIFO. Esta estrutura tem tamanho fixo e representa o formato comum das mensagens trocadas entre processos.

Os principais campos da mensagem são: `type`, que identifica o tipo de pedido, como `SUBMIT`, `STATUS` ou `SHUTDOWN`; `sender_pid`, usado para identificar o processo `runner` que enviou a mensagem; `user_id` e `command_id`, que identificam o utilizador e o comando; e `payload`, onde é enviada a *string* do comando ou outra informação necessária.

A utilização de mensagens de tamanho fixo simplifica a leitura no `controller`, pois este pode ler sempre `sizeof(RpcMessage)`. Assim, evita-se a necessidade de reconstruir mensagens de tamanho variável a partir de várias leituras parciais.

4.3. Escritas atômicas no FIFO

Como vários *runners* podem escrever no FIFO do `controller` ao mesmo tempo, foi necessário garantir que as mensagens não se misturam. A solução aproveita a garantia POSIX associada a `PIPE_BUF`: escritas para um *pipe* ou FIFO com tamanho menor ou igual a `PIPE_BUF` são atômicas.

Para manter esta propriedade, o tamanho da estrutura `RpcMessage` foi definido de forma a ficar abaixo de `PIPE_BUF`, que representa o limite até ao qual POSIX garante atomicidade em escritas para *pipes* e FIFOs. Embora em Linux este limite seja normalmente superior (frequentemente 4096 bytes), esta escolha torna o protocolo mais portátil e robusto, garantindo que cada mensagem enviada por um `runner` é lida de forma íntegra pelo `controller`.

Desta forma, cada `runner` envia a sua mensagem com uma única chamada `write()`, mantendo a integridade das mensagens mesmo quando existem vários processos a submeter pedidos em simultâneo. Esta decisão foi validada nos testes de *stress*, onde foram lançados 150

comandos concorrentes e a consulta de estado continuou a apresentar todos os pedidos corretamente.

5. Estrutura de dados e políticas de escalonamento

5.1. Representação dos comandos em espera e em execução

Para gerir o estado dos comandos, o `controller` mantém duas filas principais: uma fila de comandos em espera e uma fila de comandos em execução. Ambas são implementadas com listas ligadas e expostas através de um tipo opaco, `job_queue_t`.

Cada nó da lista representa um comando submetido e guarda a informação necessária ao seu acompanhamento, como o `command_id`, o `user_id`, o estado do processo, o PID do processo `runner` e o instante de submissão (`start_time`) para o cálculo da duração. A utilização de um tipo opaco em `scheduler.h` permite esconder a representação interna da fila. Assim, o `controller` não manipula diretamente os ponteiros da lista, usando apenas funções da API, como `queue_enqueue` e `queue_remove`. Esta separação reduz o risco de erros de manipulação de memória e torna o código mais fácil de manter.

5.2. Implementação das políticas FCFS, Random e Fair

A passagem de um comando da fila de espera para a fila de execução ocorre sempre que o número de comandos em execução é inferior ao limite de paralelismo definido no arranque do `controller`. A escolha do próximo comando depende da política de escalonamento selecionada.

Na política FCFS, os comandos são executados pela ordem de chegada. Como cada novo pedido é inserido no fim da fila, basta remover o elemento que está na cabeça da lista. Esta operação tem custo constante, $O(1)$, e garante que o comando que está há mais tempo em espera é o primeiro a ser autorizado. A justiça garantida por esta política baseia-se na ordem global de chegada ao `controller`, não sendo efetuada uma alternância explícita entre diferentes utilizadores.

Na política *Random*, o próximo comando é escolhido aleatoriamente entre os comandos em espera. Para isso, o `controller` gera um índice com `rand()` e percorre a lista até esse elemento, removendo-o da fila de espera e movendo-o para a fila de execução. Neste caso, a remoção tem custo $O(N)$, devido à necessidade de percorrer a lista ligada até ao índice escolhido. Esta política foi incluída sobretudo como termo de comparação com a política FCFS durante a avaliação experimental. Devido à natureza aleatória da escolha, a ordem apresentada na secção *Scheduled* de uma consulta de estado não representa necessariamente a ordem futura de execução.

Foi ainda implementada uma política *Fair*, baseada na alternância entre utilizadores. O `controller` mantém a ordem dos utilizadores que já submeteram comandos e guarda também o último utilizador escalonado. Quando existe uma vaga, percorre essa ordem de forma circular, ignorando utilizadores sem comandos pendentes, e escolhe o primeiro comando do próximo utilizador com trabalho em espera. Desta forma, se um utilizador tiver vários comandos em espera

e outro utilizador `submiter`, entretanto, um comando, este segundo utilizador não fica necessariamente bloqueado atrás de todos os pedidos anteriores. Esta política não interrompe comandos já iniciados, atua apenas na escolha do próximo comando a autorizar. Na opção `-c`, a secção *Scheduled* é construída a partir de uma simulação sobre um *snapshot* da fila, permitindo apresentar uma ordem coerente com a política *Fair* sem modificar a fila real.

5.3. Consulta de estado com a opção `-c`

A opção `-c` permite consultar os comandos em execução e em espera. Neste caso, o `runner` envia um pedido ao `controller`, que constrói uma resposta com o estado atual das suas filas e a envia de volta ao processo que fez a consulta.

Para evitar depender de um único buffer de tamanho fixo, o `controller` obtém um *snapshot* das filas em *arrays* alocados dinamicamente com `malloc`. A informação é depois enviada ao `runner` através de múltiplas mensagens `STATUS_RESP` de tamanho fixo. Esta abordagem permite listar um número variável de comandos, mantendo a compatibilidade com o protocolo de mensagens atômicas usado na comunicação por FIFO.

A resposta inclui primeiro os comandos em execução e depois os comandos em espera. Nos testes de stress, esta abordagem permitiu consultar corretamente o estado do sistema mesmo com 150 comandos submetidos, validando que a alocação dinâmica e a segmentação da resposta eram suficientes para cargas elevadas.

No caso da política *Fair*, os comandos em espera são apresentados segundo a ordem simulada de escalonamento, e não apenas pela ordem física da lista interna.

6. Avaliação experimental

6.1. Testes automatizados de parsing e redirecionamento

Para validar o módulo de execução e a configuração dos descritores de ficheiro, foi criada uma suite de testes automatizados em Bash. O script `test_parsing.sh` submete comandos com redirecionamentos de entrada, saída e erro (`<`, `>`, `2>`) e *pipes* (`|`), incluindo casos em que os operadores aparecem sem espaços à volta, como `echo OlaMundo>test_out.txt`, `cat<test_in.txt|wc -l>test_count.txt` e `ls ficheiro_que_ nao_existe 2>test_error.txt`.

O script valida automaticamente que o redirecionamento de saída produz o texto esperado, que a pipeline com redirecionamento conta corretamente as três linhas do ficheiro de entrada, e que o redirecionamento de erro captura a mensagem produzida pelo comando `ls`. O sucesso deste teste confirma que o *parser* consegue separar corretamente comandos, argumentos e operadores, e que o executor aplica `dup2()` e `pipe()` de forma adequada, incluindo o redirecionamento do descritor `STDERR_FILENO`.

6.2. Testes de concorrência e políticas de escalonamento

O comportamento do sistema e das políticas de escalonamento foi validado com *scripts* específicos. O *script* `test_fcfs.sh` inicia o `controller` com limite de 2 comandos em paralelo e submete 5 trabalhos idênticos, validando automaticamente que os dois primeiros comandos formam a primeira vaga de execução e que os restantes permanecem em espera pela ordem FCFS. Como comandos da mesma vaga podem terminar por ordem ligeiramente diferente, a validação do registo (*log*) é feita por grupos de execução. A política *Random* foi validada separadamente com o `test_random.sh`, executado em várias rondas com limite de concorrência de valor 1, verificando que todos os comandos são executados sem perdas nem duplicados e que surgem ordens de conclusão distintas da ordem FCFS. Adicionalmente, a política *Fair* foi validada pelo *script* `test_fair.sh`, confirmando automaticamente através do estado (`-c`) a correta alternância (*Round-Robin*) entre utilizadores e a prevenção de monopolização.

Para apoiar a análise, foi registado o tempo de permanência no sistema de cada comando, isto é, o tempo desde a receção do pedido pelo `controller` até à conclusão da execução pelo `runner`. A Tabela 1 apresenta uma execução experimental complementar, extraída do ficheiro de registo, comparando FCFS e *Random* com limite de concorrência de valor 2.

Ordem de conclusão	FCFS (lim.: 2)	Turnaround	Random (lim.: 2)	Turnaround
1.º e 2.º	User 2 / User 5	4015 / 4016 ms	User 2 / User 1	4014 / 4014 ms
3.º e 4.º	User 1 / User 4	8034 / 8036 ms	User 4 / User 3	8024 / 8029 ms
5.º	User 3	12055 ms	User 5	12040 ms

Tabela 1: Turnaround time de 5 comandos idênticos (*sleep 4*) submetidos concorrentemente.

A análise dos resultados permite observar dois aspetos importantes.

Em primeiro lugar, o limite de paralelismo está a ser respeitado. Como cada comando demora cerca de 4 segundos a executar e o `controller` permite apenas 2 comandos em paralelo, os tempos aparecem em vagas de aproximadamente 4000 ms. Os dois primeiros comandos terminam ao fim de cerca de 4 segundos. Os dois seguintes permanecem primeiro em espera e terminam ao fim de cerca de 8 segundos. O último comando só é executado depois de libertar uma nova vaga, terminando ao fim de cerca de 12 segundos.

Em segundo lugar, observa-se a diferença entre as políticas de escalonamento. Na configuração FCFS, os comandos foram autorizados de acordo com a ordem de entrada na fila do `controller`. Na configuração *Random*, a escolha dos comandos seguintes não seguiu necessariamente a ordem de chegada, mostrando que o `controller` consegue remover elementos de posições intermédias da fila de espera. Assim, estes testes validam o limite de concorrência, a ordem por vagas da política FCFS e o comportamento não determinístico da política *Random*.

6.3. Teste de stress e análise do turnaround time

O teste de stress foi realizado com o script `test_stress.sh`, que submete 150 comandos concorrentes ao `controller`. O sistema conseguiu manter os pedidos corretamente registados, sem bloqueios aparentes nem perda de mensagens. A consulta com a opção `-c` devolveu os 150 comandos esperados, validando a alocação dinâmica dos *snapshots* e o envio segmentado da resposta de estado.

Este teste também reforçou a decisão de manter a estrutura `RpcMessage` abaixo do limite mínimo de `PIPE_BUF`, garantindo que cada mensagem enviada para o FIFO do `controller` cabe numa única escrita atômica.

Por fim, o ficheiro `log.txt` foi usado para analisar os tempos registados pelo `controller`. O valor registado corresponde ao tempo decorrido entre a receção do pedido pelo `controller` e a conclusão do comando pelo `runner`, ou seja, uma forma de *turnaround time* do sistema. Esta medição foi implementada com `gettimeofday()`, conforme sugerido no enunciado do projeto. Nos testes, comandos iguais apresentaram tempos crescentes quando permaneceram mais tempo em espera, o que é coerente com o funcionamento de uma fila de escalonamento.

6.4. Metodologia de desenvolvimento dos testes com apoio de IA

De acordo com as diretrizes da Unidade Curricular, a utilização de ferramentas de Inteligência Artificial Generativa foi limitada ao apoio na criação dos testes automatizados em Bash. A lógica principal do projeto e o uso das chamadas ao sistema foram implementados pelo grupo, ficando a IA limitada à geração de *scripts* de teste a partir de requisitos definidos por nós.

Por exemplo, o script `test_parsing.sh` foi gerado a partir de uma instrução que pedia a validação automática de redirecionamentos, *pipes* e operadores sem espaços, incluindo verificação dos ficheiros produzidos, terminação controlada do servidor e limpeza robusta com `trap` (o *prompt* integral pode ser consultado no Anexo A).

Após a geração, o *script* foi revisto e executado pelo grupo, de modo a confirmar que testava os comportamentos pretendidos e que não introduzia chamadas interativas ou bloqueios inesperados. O código dos *scripts* e os *prompts* usados para os restantes testes encontram-se nos Anexos.

7. Conclusão

7.1. Síntese dos resultados obtidos

O desenvolvimento do Ambiente Multi-Runner permitiu cumprir os principais requisitos definidos para o projeto. A solução implementa um modelo cliente-servidor, separando a gestão do escalonamento, realizada pelo `controller`, da execução efetiva dos comandos, realizada pelos processos `runner`.

A implementação do *parser* e a utilização direta de chamadas ao sistema como `fork`, `execvp`, `dup2` e `pipe` permitiram suportar redirecionamentos e *pipelines* sem recorrer a `system()`

nem à execução indireta através da *shell*. A comunicação por FIFOs, juntamente com a limitação do tamanho da `RpcMessage` para respeitar o limite mínimo de `PIPE_BUF`, permitiu manter mensagens atômicas mesmo com vários *runners* a submeter pedidos em simultâneo.

Foram também implementadas três políticas de escalonamento, FCFS, *Random* e *Fair*, apoiadas por estruturas de dados encapsuladas. A opção `-c` permitiu consultar os comandos em execução e em espera, e os testes automatizados validaram o comportamento do sistema em cenários de *parsing*, concorrência, justiça entre utilizadores e stress.

7.2. Limitações e possíveis melhorias

A principal limitação da solução está no tamanho fixo do *payload* da `RpcMessage`. Esta opção foi tomada para garantir que cada mensagem cabe numa escrita atômica para o FIFO, mas limita o comprimento máximo dos comandos submetidos. Uma melhoria futura passaria por implementar um protocolo de mensagens fragmentadas, com reconstrução correta no `controller`.

A implementação da política *Fair* já mitiga alguns problemas de justiça no escalonamento, especialmente em cenários onde um utilizador submete muitos comandos antes dos restantes. Ainda assim, esta política atua apenas no momento de escolher o próximo comando a autorizar. Uma melhoria futura seria enriquecer a política *Fair* com métricas adicionais, por exemplo considerando tempos médios de espera por utilizador ou prioridades configuráveis. Outra evolução, mais complexa, seria estudar mecanismos de *time-sharing*, mas isso exigiria interromper e retomar processos, alterando significativamente a arquitetura atual

Anexo A – Teste de parsing e redireccionamentos

Instrução fornecida à ferramenta de IA para a geração do script:

```
Escreve um script em Bash (test_parsing.sh) para validar o comportamento de um parser manual em C.  
O script deve:  
1. Iniciar o servidor (./bin/controller 2 fcfs) em background e guardar o seu PID  
2. Submeter comandos usando o nosso cliente (./bin/runner) onde os redireccionamentos de entrada, saída e erro, bem como os pipes, estejam colados aos argumentos, sem espaços (ex: echo OlaMundo>test_out.txt, cat<test_in.txt|wc -l>test_count.txt e ls ficheiro_que_nao_existe 2>test_error.txt)  
3. Verificar automaticamente que test_out.txt contém o texto esperado, que test_count.txt contém a contagem correta de linhas e que test_error.txt contém a mensagem de erro gerada pelo comando ls  
4. Enviar um pedido de terminação (-s) e fazer wait pelo PID do servidor  
O script deve ser robusto, falhar com mensagem clara quando algum resultado não corresponde ao esperado, e usar trap para limpar os ficheiros temporários ao terminar.
```

Anexo B – Teste da política FCFS e limite de concorrência

Instrução fornecida à ferramenta de IA para a geração do script:

Escreve um script em Bash (test_fcfs.sh) para validar a política FCFS do meu orquestrador em C.

O script deve iniciar o controller com limite de concorrência 2 e política fcfs, submeter 5 comandos sleep em background com pequenas pausas entre submissões para tornar a ordem de chegada previsível, consultar o estado com -c e validar automaticamente que os utilizadores 1 e 2 estão na primeira vaga de execução e que os utilizadores 3, 4 e 5 aparecem na secção Scheduled por esta ordem.

Depois deve encerrar com -s, esperar pelos runners e validar no log.txt que os comandos terminaram por vagas compatíveis com o limite de concorrência.

Anexo C – Teste de stress ao canal IPC

Instrução fornecida à ferramenta de IA para a geração do script:

Cria um script em Bash (`test_stress.sh`) para realizar um teste de carga ao meu sistema de FIFOs em C.

O objetivo é validar que as mensagens enviadas para o FIFO se mantêm íntegras, tirando partido da garantia de atomicidade associada a `PIPE_BUF`, e que a construção dinâmica da resposta da opção `-c` funciona corretamente. O script deve iniciar o controller com limite de concorrência 10 e submeter 150 runners quase em simultâneo, usando `&` e uma pequena pausa intermitente para evitar sobrecarregar momentaneamente o sistema com demasiados processos criados ao mesmo tempo.

A seguir, deve guardar o resultado da opção `-c` num ficheiro e usar comandos como `grep` e `wc` para verificar automaticamente se o ficheiro contém exatamente as 150 linhas correspondentes às tarefas submetidas.

Anexo D – Teste da política de escalonamento Fair

Instrução fornecida à ferramenta de IA para a geração do script:

Cria um script em Bash (test_fair.sh) para validar a política de escalonamento fair (Round-Robin por utilizador).

O script deve:

1. Iniciar o controller em background com limite de concorrência 1 e a política fair (`./bin/controller 1 fair`).
2. Submeter rapidamente 3 comandos `sleep 2` do utilizador 1 em background e, logo a seguir, um comando do utilizador 2.
3. Capturar o estado do sistema com a opção `-c` para um ficheiro temporário.
4. Usar `awk` para processar o ficheiro gerado e verificar automaticamente se o utilizador 2 é o primeiro a aparecer na secção `Scheduled` ou se já se encontra em `Executing`, validando que a política aplicou a alternância entre utilizadores e não o manteve atrás de todos os comandos anteriores do utilizador 1.
5. Terminar com `-s` e limpar os processos filhos e ficheiros temporários usando `trap`.

Anexo E – Teste da política Random

Instrução fornecida à ferramenta de IA para a geração do script:

```
Cria um script em Bash (test_random.sh) para validar a política random do
orquestrador em C.
O script deve executar várias rondas com o controller configurado com
limite de concorrência 1 e política random.
Em cada ronda deve submeter 5 comandos sleep 1, encerrar o controller com -
s, analisar o log.txt e verificar que os utilizadores 1 a 5 aparecem
exatamente uma vez, sem perdas nem duplicados.
O script deve ainda confirmar que, ao longo das várias rondas, surge pelo
menos uma ordem diferente da ordem FCFS, evidenciando que a escolha
aleatória está a ser aplicada.
```